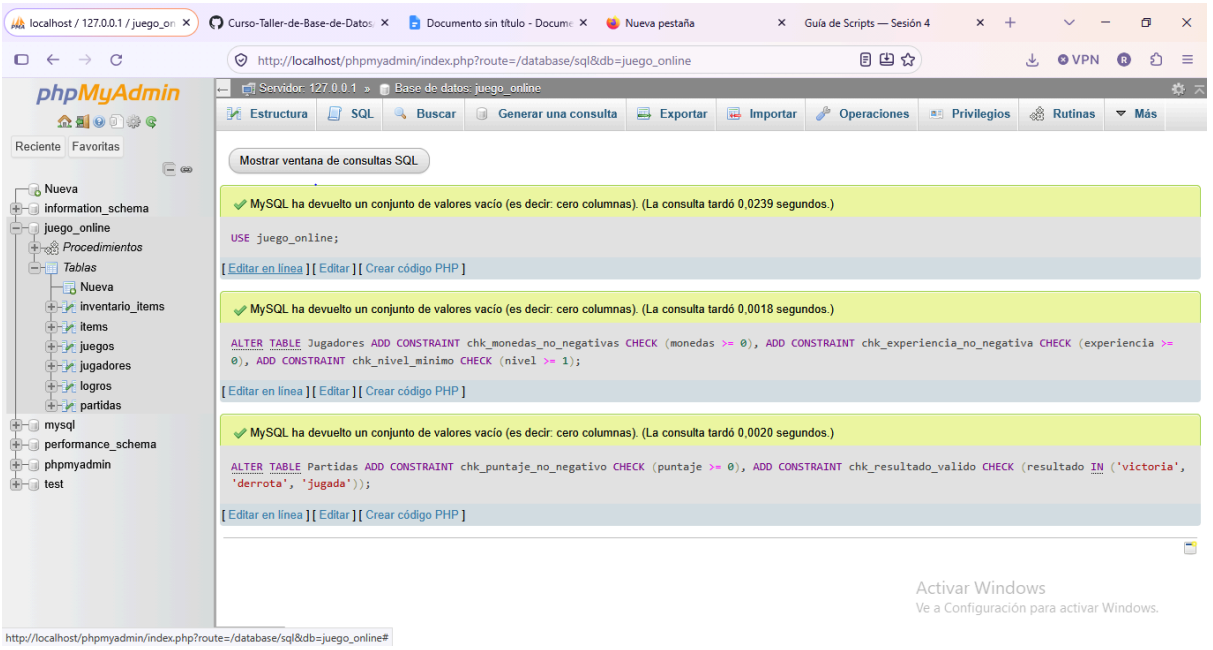
1. Integridad estructural y semántica

None

```
USE juego_online;
ALTER TABLE Jugadores
    ADD CONSTRAINT chk_monedas_no_negativas CHECK (monedas >= 0),
    ADD CONSTRAINT chk_experiencia_no_negativa CHECK (experiencia >= 0),
    ADD CONSTRAINT chk_nivel_minimo CHECK (nivel >= 1);
ALTER TABLE Partidas
    ADD CONSTRAINT chk_puntaje_no_negativo CHECK (puntaje >= 0),
    ADD CONSTRAINT chk_resultado_valido CHECK (resultado IN ('victoria', 'derrota',
'jugada'));
```



esta consulta agrega restricciones a las tablas jugadores y partidas para evitar que se registren datos incorrectos, esta agrega 5 restricciones de la bd donde si se intenta ingresar un dato que no umple estas condiciones esta rechaza la operacion por lo tanto el codigo tiene como proposito proteger informacion del juego evitando valores negativos y resultados en la tablas no permitidos.

2. Roles de aplicación: jugador, moderador, administrador:

None

```
USE juego_online;

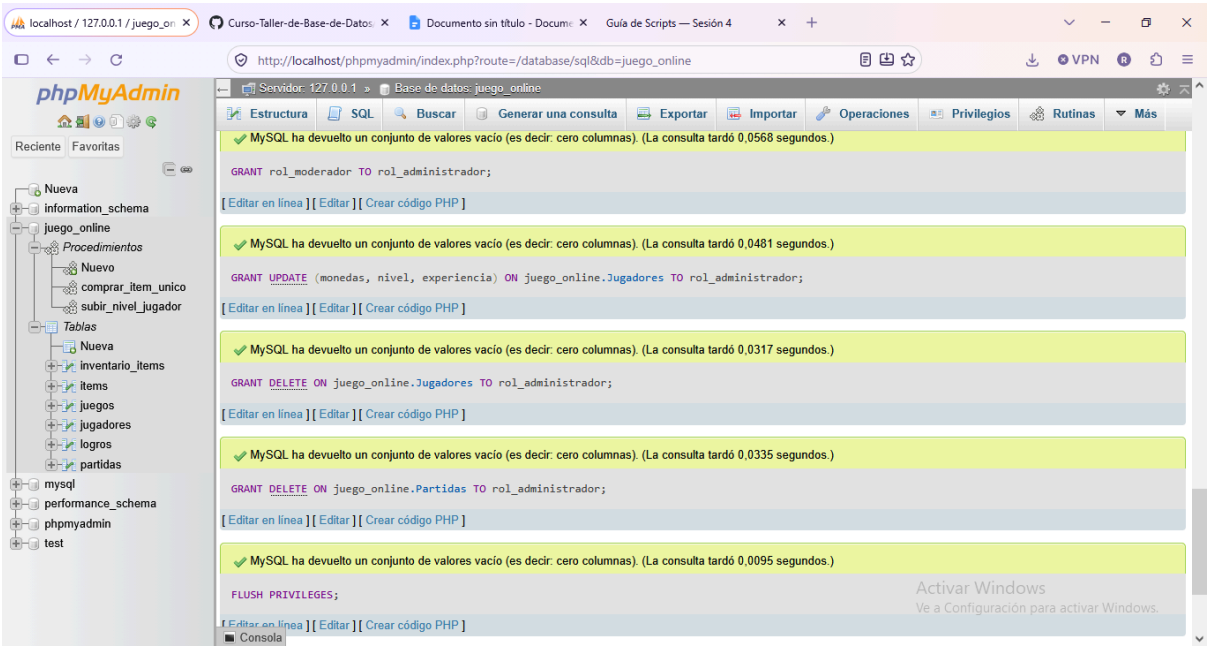
DROP ROLE IF EXISTS rol_jugador;
CREATE ROLE rol_jugador;
GRANT SELECT ON juego_online.Jugadores TO rol_jugador;
GRANT SELECT ON juego_online.Items TO rol_jugador;
GRANT SELECT ON juego_online.Juegos TO rol_jugador;
GRANT SELECT, INSERT ON juego_online.Partidas TO rol_jugador;
GRANT EXECUTE ON PROCEDURE juego_online.subir_nivel_jugador TO rol_jugador;
GRANT EXECUTE ON PROCEDURE juego_online.comprar_item_unico TO rol_jugador;

DROP ROLE IF EXISTS rol_moderador;
CREATE ROLE rol_moderador;
GRANT rol_jugador TO rol_moderador;   GRANT UPDATE (baneado) ON juego_online.Jugadores TO rol_moderador;

DROP ROLE IF EXISTS rol_administrador;
CREATE ROLE rol_administrador;
```

```
GRANT rol_moderador TO rol_administrador;
GRANT UPDATE (monedas, nivel, experiencia) ON juego_online.Jugadores TO rol_administrador;
GRANT DELETE ON juego_online.Jugadores TO rol_administrador;
GRANT DELETE ON juego_online.Partidas TO rol_administrador;

FLUSH PRIVILEGES;
```

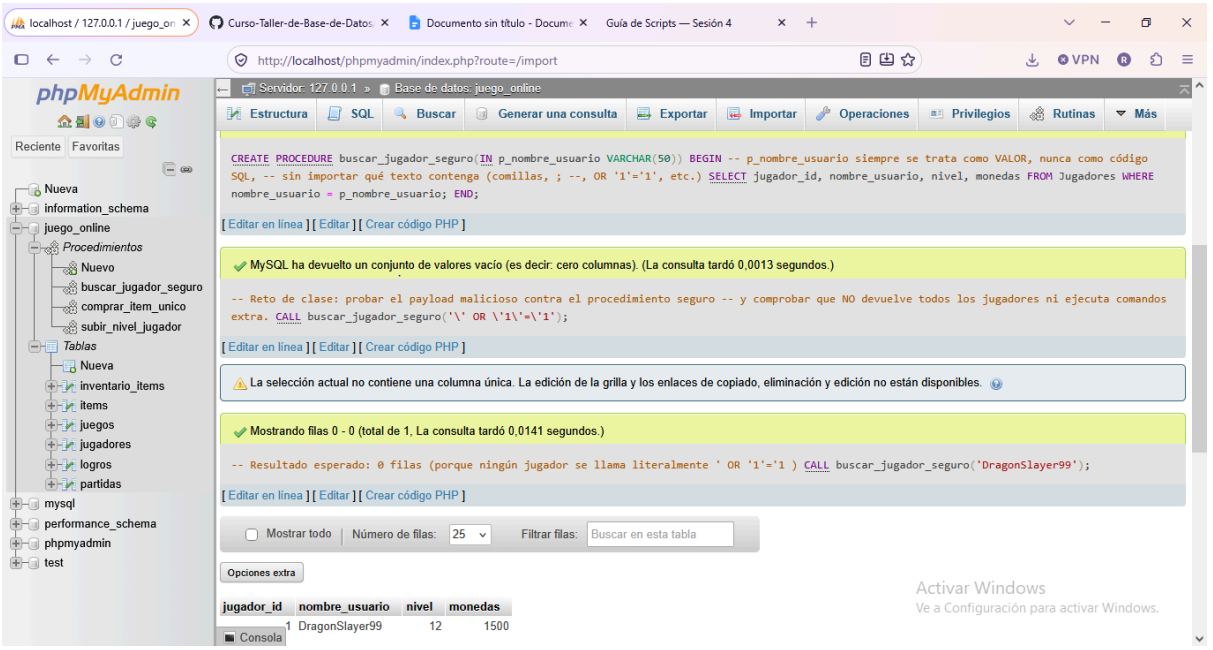


esta consulta crea 3 roles de usuario par acontrolar los permisos dentro de la bd:jugador moderador y administrador. tambien agrega una columna llamada baneado a la tabla jugadores para poer indicar sdi un jugador esta baneado o no, protegiendo asi la informacion y evitando accesos no autoriados

3. Simulador anti-inyección SQL:

None

```
USE juego_online;
DROP PROCEDURE IF EXISTS buscar_jugador_seguro;
DELIMITER //
CREATE PROCEDURE buscar_jugador_seguro(IN p_nombre_usuario VARCHAR(50))
BEGIN
     SELECT jugador_id, nombre_usuario, nivel, monedas
    FROM Jugadores
    WHERE nombre_usuario = p_nombre_usuario;
END //
DELIMITER ;
-- Resultado esperado: 0 filas (porque ningún jugador se llama literalmente  ' OR '1'='1 )
CALL buscar_jugador_seguro('DragonSlayer99');
```



esta consulta crea un procedimiento llamado buscar jugador seguro par poder buscar jugadores por su nombre y asi proteger la bd de contra ataques de inyección sql